# Distribution and Scalability

Khoa A. Vu & Kelly Pham

Student ID: 030063200 & 03039600

California State University, Long Beach

CECS 327 - Section 01

Dr. Hailu Xu

December 05, 2025

**Objective**

In this lab, we are expanding on our P2P network to support decentralized storage and retrieval. Each node will be able to:

- Upload and download files
- Insert and query key-value pairs
- Distribute storage responsibilities using a basic Distributed Hash Table

**Implementation**

To start off, we made a storage directory for the files that would be uploaded when testing and for retrieving. This was done by os.path.join(os.path.dirname(__file__), "storage"). After the files are uploaded we used request.files to retrieve the files and secure_filename to ensure that the name is saved into the storage that we created. Like how we created an upload endpoint, we also needed to create the download endpoint where users could retrieve the files. This was possible through send_from_directory where it takes the file from the storage folder. For the key value pairs, we created a kv_store in order to be able to store the pairs. We implemented POST since users will need to be able to add the key value pairs and GET for users to retrieve. We used SHA1 to start off building our distributed hashing table and the build_ring function is there for us to sort the list of nodes and their corresponding hash ID. The function find_responsible_node's job is to distribute and determine responsibility. It decides which node is storing a value based on the ID and hash value. When a client sends JSON to kv_put or kv_get, this request is routed to the correct node. In the event that a node receives a request from another node, we have a forwarding_request function for this purpose. It sends a request to the

node with a key that is not its own and ensures that the storage responsibility is shared, a distributed hashing table.

When working on this lab, some implications we ran into were mounting the file upload/download. We had to ensure that the files paths matched inside the containers and remained that way when restarting Docker. If they did not match, we would get errors such as “File not found” which prevented us from moving further. Additionally, there were some issues with the DHT ring construction. The Distributed Hashing Table required components such as Hashing node URLs with SHA-1, sorting nodes based on hash values, and determining the correct “responsible node”. As a result, some errors that we ran into were keys being stored on the same node, incorrect forwarding, and sometimes even infinite routing loops.

**Screenshots**

```python
import os # working with paths and environment variables.
import hashlib # computing SHA-1 hashes (used to map nodes and keys onto the DHT ring).
from typing import Dict, List # type hints to make code clearer (Dict[str, str], etc.).
import requests # to send HTTP requests to other nodes (forwarding).

from flask import Flask, request, jsonify, send_from_directory
# flask imports:
# Flask – main application class.
# request – incoming HTTP request object.
# jsonify – easy way to return JSON responses.
# send_from_directory – helper to send files from a folder.

from werkzeug.utils import secure_filename # sanitizes file names so users can't inject weird paths.

import logging
from datetime import datetime

# Logging Setup
class LogFormat(logging.Formatter):
    """Custom log formatter with timestamps."""
    def format(self, record):
        timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
        level = record.levelname
        message = record.getMessage()
        return f"[{timestamp}] [{level}] {message}"


logger = logging.getLogger("DHT_Node")
logger.setLevel(logging.INFO)

_stream_handler = logging.StreamHandler()
_stream_handler.setFormatter(LogFormat())
# Avoid adding multiple handlers if app.py is reloaded
if not logger.handlers:
    logger.addHandler(_stream_handler)

def log_info(msg: str) -> None:
    logger.info(msg)

def log_warn(msg: str) -> None:
    logger.warning(msg)

def log_error(msg: str) -> None:
    logger.error(msg)
```

```python
# Configuration

# Base directory for file storage
STORAGE_DIR = os.path.join(os.path.dirname(__file__), "storage")

# Ensure storage directory exists
os.makedirs(STORAGE_DIR, exist_ok=True)

# Node identity and peer configuration via environment variables
# this node's own URL (e.g., http://node1:5000), read from environment;
# Default if not set: http://localhost:5000.
SELF_URL = os.getenv("SELF_URL", "http://localhost:5000")

# raw comma-separated string of all peer URLs (including self), e.g.
# "http://node1:5000,http://node2:5000,http://node3:5000"
PEERS_ENV = os.getenv("PEERS", "")


def parse_peers(peers_env: str) -> List[str]:
    """
    Convert the raw PEERS environment variable into a clean, sorted list.

    Example:
        "http://node1:5000, http://node2:5000" ->
        ["http://node1:5000", "http://node2:5000"]
    """
    peers = [p.strip() for p in peers_env.split(",") if p.strip()]
    # Ensure self is included (optional, but convenient)
    if SELF_URL not in peers:
        peers.append(SELF_URL)
    unique_peers = sorted(set(peers))
    log_info(f"Configured peers: {unique_peers}")
    return unique_peers

PEERS: List[str] = parse_peers(PEERS_ENV)

# DHT Helper Functions
def sha1_to_int(value: str) -> int:
    """Return SHA-1 hash of a string as an integer."""
    h = hashlib.sha1(value.encode("utf-8")).hexdigest()
    return int(h, 16)  # a big integer representing the hash position on the ring.

def build_ring(peers: List[str]) -> List[Dict]:
    """
    Build a sorted list of nodes on the ring.
    Each element: {"url": url, "id": int_hash}
    """
```

```python
    ring = [{"url": url, "id": sha1_to_int(url)} for url in peers]
    ring.sort(key=lambda n: n["id"])
    log_info(f"DHT ring built with nodes: {ring}")
    return ring

RING = build_ring(PEERS)

def find_responsible_node(key: str) -> Dict:
    """
    Given a key, find the node dict responsible for it based on the ring.

    The algorithm:
        1. Hash the key to an integer.
        2. Walk the ring in ascending order and return the first node whose
           ID is >= key hash.
        3. If none, wrap around and return the first node.
    """
    key_id = sha1_to_int(key)
    for node in RING:
        if key_id <= node["id"]:
            return node
    # wrap-around
    return RING[0]

def is_local_node(node: Dict) -> bool:
    """Check if the given node dict corresponds to this running node."""
    return node["url"] == SELF_URL

# In-memory Key-Value Store
kv_store: Dict[str, str] = {}

# Flask App
app = Flask(__name__)
```

```python
# Health & Peer Info
@app.route("/health", methods=["GET"])
def health():
    """Basic health check endpoint."""
    log_info(f"GET /health from {request.remote_addr}")
    return jsonify(
        {
            "status": "ok",
            "self": SELF_URL,
            "peers": PEERS,
        }
    )

@app.route("/peers", methods=["GET"])
def get_peers():
    """Return the peer list and ring for debugging."""
    log_info(f"GET /peers from {request.remote_addr}")
    return jsonify(
        {
            "self": SELF_URL,
            "peers": PEERS,
            "ring": RING,
        }
    )

# File Upload & Download
@app.route("/upload", methods=["POST"])
def upload_file():
    """
    Upload a file to this node's local storage directory.
    Expects form-data: file=@filename
    """
    log_info(f"POST /upload from {request.remote_addr}")

    if "file" not in request.files:
        log_warn("Upload failed - no file part in request")
        return jsonify({"error": "No file part"}), 400

    f = request.files["file"]
    if f.filename == "":
        log_warn("Upload failed - empty filename")
        return jsonify({"error": "No selected file"}), 400

    filename = secure_filename(f.filename)
    save_path = os.path.join(STORAGE_DIR, filename)
    f.save(save_path)
```

```python
    log_info(f"File uploaded: {filename} -> {save_path}")

    return jsonify(
        {
            "status": "ok",
            "filename": filename,
            "stored_at": save_path,
            "node": SELF_URL,
        }
    )


@app.route("/download/<path:filename>", methods=["GET"])
def download_file(filename):
    """
    Download a file from this node's local storage directory.
    """
    log_info(f"GET /download/{filename} from {request.remote_addr}")

    full_path = os.path.join(STORAGE_DIR, filename)
    if not os.path.exists(full_path):
        log_warn(f"File not found: {filename}")
        return jsonify({"error": "File not found"}), 404

    log_info(f"File sent: {filename}")
    # Security is minimal here; in a real system you'd be stricter
    return send_from_directory(STORAGE_DIR, filename, as_attachment=True)


# Helper: Forwarding Requests
def forward_request(node_url: str, method: str, path: str, **kwargs):
    """
    Forward an HTTP request to another node.

    Args:
        node_url: Base URL of the destination node (e.g. http://node2:5000).
        method: "GET" or "POST".
        path: Route path starting with '/', e.g. "/kv" or f"/kv/{key}".
        **kwargs: Passed directly to requests.get/post (json=..., params=..., etc.).

    Returns:
        (json_body, status_code)
    """
    url = node_url.rstrip("/") + path
    log_info(f"Forwarding {method.upper()} {path} to {node_url}")
```

```python
    try:
        if method.upper() == "GET":
            resp = requests.get(url, timeout=5, **kwargs)
        elif method.upper() == "POST":
            resp = requests.post(url, timeout=5, **kwargs)
        else:
            raise ValueError(f"Unsupported method {method}")

        # We assume JSON response from peer
        return resp.json(), resp.status_code

    except requests.RequestException as e:
        log_error(f"Failed to contact node {node_url}: {e}")
        return {"error": f"Failed to contact node {node_url}", "details": str(e)}, 502


# Key-Value Endpoints with DHT Routing
@app.route("/kv", methods=["POST"])
def kv_put():
    """
    Store a key-value pair in the DHT.

    Expects JSON body:
        { "key": "...", "value": "..." }

    The responsible node is chosen using the DHT ring. If this node is not
    responsible for the key, it will forward the request to the correct node.
    """
    data = request.get_json(silent=True) or {}
    key = data.get("key")
    value = data.get("value")

    log_info(f"POST /kv from {request.remote_addr} - key={key}, value={value}")

    if key is None or value is None:
        log_warn("KV PUT failed - JSON missing 'key' or 'value'")
        return jsonify({"error": "JSON must contain 'key' and 'value'"}), 400

    responsible = find_responsible_node(key)
    log_info(f"Key '{key}' is mapped to node {responsible['url']}")
```

```python
    if is_local_node(responsible):
        # Handle locally
        kv_store[key] = value
        log_info(f"Stored key locally: {key} -> {value}")
        return jsonify(
            {
                "status": "stored",
                "key": key,
                "value": value,
                "node": SELF_URL,
            }
        )
    else:
        # Forward to responsible node
        log_info(f"Forwarding key '{key}' to {responsible['url']}")
        json_resp, status = forward_request(responsible["url"], "POST", "/kv", json=data)
        return (
            jsonify(
                {
                    "forwarded_to": responsible["url"],
                    "original_node": SELF_URL,
                    "response": json_resp,
                }
            ),
            status,
        )


@app.route("/kv/<key>", methods=["GET"])
def kv_get(key):
    """
    Retrieve a key-value pair from the DHT.

    The responsible node is chosen using the DHT ring. If this node is not
    responsible, it forwards the request to the correct node.
    """
    log_info(f"GET /kv/{key} from {request.remote_addr}")

    responsible = find_responsible_node(key)
    log_info(f"Key '{key}' is mapped to node {responsible['url']}")
```

```python
    if is_local_node(responsible):
        if key in kv_store:
            value = kv_store[key]
            log_info(f"Returned local value: {key} = {value}")
            return jsonify(
                {
                    "status": "found",
                    "key": key,
                    "value": value,
                    "node": SELF_URL,
                }
            )
        else:
            log_warn(f"Key '{key}' not found on local node")
            return jsonify(
                {
                    "status": "not_found",
                    "key": key,
                    "node": SELF_URL,
                }
            ), 404
    else:
        log_info(f"Forwarding GET /kv/{key} to {responsible['url']}")
        json_resp, status = forward_request(
            responsible["url"], "GET", f"/kv/{key}"
        )
        return (
            jsonify(
                {
                    "forwarded_to": responsible["url"],
                    "original_node": SELF_URL,
                    "response": json_resp,
                }
            ),
            status,
        )


if __name__ == "__main__":
    port = int(os.getenv("PORT", "5000"))
    log_info(f"Starting DHT node at {SELF_URL} on port {port}")
    # Listen on all interfaces so Docker can expose the port
    app.run(host="0.0.0.0", port=port, debug=False)
```

*app.py - Acts as a central registry. Keeps a list of all registered active peers.*

```dockerfile
# Simple Dockerfile for a single P2P/DHT node
FROM python:3.11-slim

# Install basic system deps (optional, but often useful)
RUN apt-get update && apt-get install -y --no-install-recommends \
    curl \
 && rm -rf /var/lib/apt/lists/*

WORKDIR /app

# Copy requirements and install
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy application code
COPY app.py .

# Storage directory inside container
RUN mkdir -p /app/storage

# Environment defaults (can be overridden)
ENV PORT=5000

EXPOSE 5000

CMD ["python", "app.py"]
```

*Dockerfile - Builds the Docker image for all peer nodes.*

```yaml
version: "3.9"

Run All Services
services:
  Run Service
  node1:
    build: .
    container_name: node1
    environment:
      - SELF_URL=http://node1:5000
      - PEERS=http://node1:5000,http://node2:5000,http://node3:5000
      - PORT=5000
    ports:
      - "5001:5000"
    volumes:
      - node1_storage:/app/storage

  Run Service
  node2:
    build: .
    container_name: node2
    environment:
      - SELF_URL=http://node2:5000
      - PEERS=http://node1:5000,http://node2:5000,http://node3:5000
      - PORT=5000
    ports:
      - "5002:5000"
    volumes:
      - node2_storage:/app/storage

  Run Service
  node3:
    build: .
    container_name: node3
    environment:
      - SELF_URL=http://node3:5000
      - PEERS=http://node1:5000,http://node2:5000,http://node3:5000
      - PORT=5000
    ports:
      - "5003:5000"
    volumes:
      - node3_storage:/app/storage

volumes:
  node1_storage:
  node2_storage:
  node3_storage:
```

*docker-compose.yml - The main logic for each peer — registers itself, discovers others, and sends/receives messages.*

## Contributions

- Kelly Pham: Writing up the project report and extra credit option
- Khoa Vu: Worked on the code, Docker Compose setup, while also creating the README and video recording.
- Both: Reviewing task, documentation.

## Result

Execution Demonstration Video

## References

1) Pallets Projects. (n.d.). *File uploads*. Flask. Retrieved December 7, 2025, from https://flask.palletsprojects.com/en/stable/patterns/fileuploads/

2) Algodaily. (n.d.). *Designing a simple key-value store*. Retrieved December 7, 2025, from https://algodaily.com/lessons/designing-a-simple-key-value-store-af5f4c6a